

Programming & Algorithms

Aula 07: Strings

Formatação

Estruturas de seleção aninhadas

Prof. Ettore Baldini-Neto

ettore.neto@faculdadeimpacta.com.br

Aula Passada

Na aula passada aprendemos sobre operadores relacionais: ==, !=, >, <, >= e <=.

Aprendemos também sobre operadores lógicos: not, and, or.

E finalmente, começamos a estudar estruturas de seleção simples (if) e composta (if - else).

Aula de hoje

Nesta aula vamos aprender mais sobre:

- Strings e sua manipulação
- Formatação
- Também sofisticaremos a utilização de estruturas de controle (seleção) de tal modo que possamos trabalhar com problemas mais complexos. Mais especificamente, vamos falar de estruturas de seleção aninhadas

Strings

Já aprendemos que strings são usadas para controlar cadeias de caracteres (texto). Vamos aprender um pouco mais sobre como manipulá-las introduzindo algumas operações

Concatenação: é utilizada quando queremos/precisamos juntar duas ou mais strings. O operador + é utilizado.

Exemplo:

```
Type "help", "copyright", "credits" or '
>>> ' Bom ' + 'dia'+ '!'
' Bom dia!'
>>> 'Eu' + 'estou' + 'faminto' + '!'
'Euestoufaminto!'
>>> 'Eu '+ 'estou ' + 'faminto '+ '!'
'Eu estou faminto !'
>>>
```

Strings

Note que a concatenação não altera seus operandos mas gera uma nova string que inclusive pode ser atribuída a uma nova variável

```
Type "help", "copyright", "credits" or "license()" for more i
>>> nome = 'Paula'
>>> idade = 26
>>> x = 'Olá ' + nome + '! Você tem ' + str(idade) + ' anos?'
>>> print(x)
Olá Paula! Você tem 26 anos?
>>>
```

Note que a função integrada str( ) recebe qualquer valor e converte para string. Ela foi necessária para que a concatenação pudesse ser feita.

Strings

Repetição: Para gerar uma nova string que repita o conteúdo de uma string mais simples. É o operador *

Exemplo:

```
>>>
>>> riso_contido = 'haha'
>>> riso_aberto = riso_contido * 4
>>> riso_aberto
'hahahahahahahaha'
>>>
```

Strings

Comparação: para, como o nome diz, realizar comparações entre strings com a utilização de operadores relacionais.

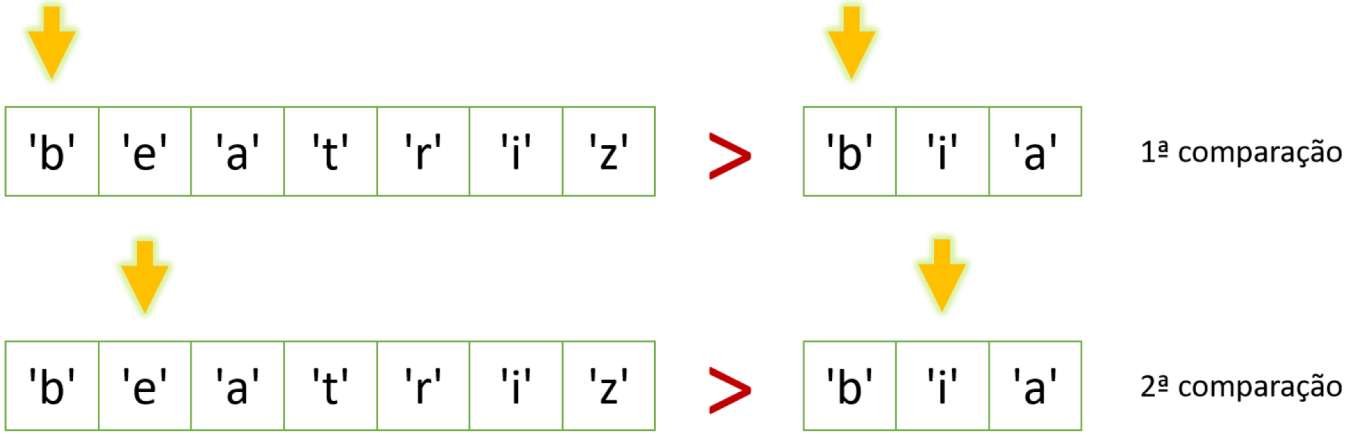
```
>>>
>>> 'maria'!= 'MARIA'
True
>>> 'maria' == 'MARIA'
False
>>> 'Paul' < 'Paulo'
True
>>> 'beatriz' > 'bia'
False
>>>
```

Atenção: Em Python a comparação entre strings é feita caractere à caractere. **Não é o tamanho da string** que diz se ela é maior ou menor do que outra mas sim seu **conteúdo!**

Strings

- O importante é que: Quando caracteres são comparados, internamente **ocorre a comparação entre os números naturais que correspondem a esses caracteres.** Estes números são códigos da **tabela unicode.**
- Unicode consiste de uma padronização que permite que computadores representem e manipulem caracteres de quase todos os sistemas de escritas existentes.

https://unicode-table.com/pt




```
>>>
>>>
>>>
>>> ord('i')
105
>>> ord('e')
101
>>>
```

Por isso que beatriz > bia -> False

```
>>>
>>>
>>> 'ANA'< 'ana'
True
>>> print('A = ', ord('A'), '| a =', ord('a'))
A =  65 | a = 97
>>>
```

A ordem seguida é a do alfabeto e letras maiúsculas são listadas **antes** das letras minúsculas.

Strings

Temos a função chr que faz o inverso da função ord, ou seja, recebe como argumento um número real (codigo unicode) e retorna o caractere correspondente.

```
>>>
>>> print(chr(48), chr(49), chr(50), chr(9733))
...
0 1 2 ★
>>>
```

Formatação

Python possui algumas formas para a formatação de strings;

1. **Interpolação:** utilizando o operador de interpolação (%). Ainda é compatível com todas as versões mas o uso é desencorajado (Caso queira saber mais, recorra ao livro Introdução à progamação em Python, N.N.C.Menezes)
2. **Método format:** método no qual os argumentos são formatados e inseridos nos marcadores identificados por pares de chaves. Para projetos Python que usam versões anteriores à 3.6

Exemplo método format.

```
>>> nome = 'João'
>>> idade = 22
>>> grana = 105.50
>>> '{} tem {} anos e R${} no bolso.'.format(nome, idade, grana)
'João tem 22 anos e R$105.5 no bolso.'
>>>
```

3. Template strings: é uma forma especial de criação de strings, sendo recomendada para casos nos quais a formatação da string é fornecida pelo usuário final. Não será abordada nesse curso.

Formatação

4. f-strings ou 'strings literais formatadas': é uma string comum prefixada com a letra f ou F antes da abertura de apóstrofos/aspas e com valores entre pares de chaves que serão formatados e inseridos na string. **É a forma que utilizaremos!**

Formatação com f-strings

Exemplos:

```
>>>
>>> mensagem = f'2 + 3 = {2 +3}, entendeu?'
...
>>> mensagem
...
'2 + 3 = 5, entendeu?'
>>>
```

```
>>> salario = float(input('Seu salário: '))
Seu salário: 1000.0
>>> f'20% a mais em R$ {salario} dará R$ {salario * 1.2}'
'20% a mais em R$ 1000.0 dará R$ 1200.0'
>>>
```

Explorando possibilidade com f-strings

Para formatar um dado do tipo float o seguinte padrão de formatação deve ser usado.

f ‘ {<valor>:<colunas>.<decimais>f}’

no qual:

- **colunas:** quantidade mínima de colunas reservadas para o valor na string formatada. Cada caractere ocupa uma coluna, inclusive o ponto.
- **decimais:** o número total de casas decimais; a última casa à direita sofre arredondamento. Caso seja omitido, o padrão é de 6 casas decimais.
- **f:** indica que a formatação será feita para o tipo float.

Explorando possibilidades com f-strings

```
>>> import math as mt
>>> mt.pi
3.141592653589793
>>> f'{mt.pi:f}' # sem especificação, o padrão é 6 casas decimais
'3.141593'
>>> f'{mt.pi:.3f}' # note que há o arredondamento na última casa decimal
'3.142'
>>> f'{mt.pi:7.3f}' # 7 colunas foram reservadas para a parte decimal
'  3.142'
>>>
```

```
>>> c_planck = 6.62607015e-36
>>> print(f'A constante de Planck é dada por {c_planck:.3e} J/s')
A constante de Planck é dada por 6.626e-36 J/s
>>> print(f'A constante de Planck é dada por {c_planck:.2e} J/s')
A constante de Planck é dada por 6.63e-36 J/s
>>>
```

Existem outras possibilidades de formatação de variáveis float e também para outros tipos como int e string**. Consulte a documentação oficial do Python.**

Explorando possibilidade com f-strings

Pode-se definir o tamanho mínimo de colunas para qualquer string formatada, independente do tipo de dado. O padrão é:

f ‘{<valor>:<colunas>}’

f ‘ o nome da menina é {<valor>:<colunas>}’

Exemplo:

```
>>> nome = 'Paula'
>>> f'{nome:10}'
'Paula     '
>>>
```

Explorando possibilidades com f-strings

Note que o alinhamento padrão utilizado para números é à direita, já para valores do tipo string o alinhamento padrão é à esquerda. É possível controlar o alinhamento adicionando um dos símbolos da tabela abaixo.

Símbolo	Alinhamento
<	à esquerda
>	à direita
^	centralizado

sintaxe: f '{<valor>:<caractere><alinhamento><colunas>}'

Alinhamento com f -strings

Exemplos:

```
>>>
>>> f'{"à esquerda":<30}'
'à esquerda                    '
>>> f'{"à direita":>30}'
'                     à direita'
>>> f'{"ao centro":^30}'
'           ao centro          '
>>>
```

Flags (bandeiras) booleanas

- Flags booleanas servem para **sinalizar** algo no código e podem ser utilizadas para facilitar a leitura das estruturas de seleção e também de repetição indefinida.
- Uma flag booleana é simplesmente uma variável que **guarda o resultado de uma comparação ou expressão relacional/lógica.**
- Logo, ao acessar o valor dessa variável ao longo do fluxo de execução do programa, temos como saber se a condição em questão foi atendida ou não.
- Pictoricamente: bandeira levantada (True) ou não (False).

Vamos ver algumas codificações que utilizam este recurso

Flags booleanas

```
idade = int(input('Qual a sua idade? '))

if idade >= 18:
    print('Você é maior de idade.')
```

O bloco de código do if só será executado quando o usuário fornecer um valor para a idade que seja maior do que 18.

Vamos reescrever este trecho utilizando uma flag booleana que vai armazenar o resultado da condição que estamos testando

Flags booleanas

```
idade = int(input('Qual sua idade? '))
maior_de_idade = idade >= 18

if maior_de_idade:
    print('Você é maior de idade.')
```

- As duas codificações são **equivalentes.** É bom ter isso em mente!
- Note que na codificação acima, **não há operadores na condição if**, apenas uma variável com valor booleano.

Flags booleanas

Exemplo:

```
idade = int(input('Qual a sua idade? '))
cnh = input('Você tem CNH? (s/n)')

if idade >= 18 and cnh == 's':
    print('Você pode dirigir.')
```

Vamos agora substituir este código pelo equivalente no qual as condições do if são substitutídas por flags booleanas.

Flags booleanas

```
idade = int(input('Qual a sua idade? '))
cnh = input('Você tem CNH? (s/n)')

maior_de_idade = idade >= 18
possui_cnh = cnh == 's'
pode_dirigir = maior_de_idade and possui_cnh

if pode_dirigir:
    print('Você pode dirigir.')
```

Flags booleanas

Vantagens da utilização:

- Melhor legibilidade dos códigos
- Armazenamento de informações relativas a uma determinada condição avaliada na execução do código
- Aprimora o desempenho do programa quando uma condição deve ser avaliada várias vezes, como em laços, por exemplo.

Atenção: Cabe a cada programador utilizar as flags de acordo com sua expertise. O objetivo principal neste momento é entender como elas funcionam.

Estruturas de seleção aninhadas

Nem sempre nossos programas serão simples e, muitas vezes, precisaremos aninhar vários if's para obter o comportamento desejado do programa. **Aninhar significa utilizar um if dentro de outro.** As regras são as mesmas, ou seja, os dois pontos :

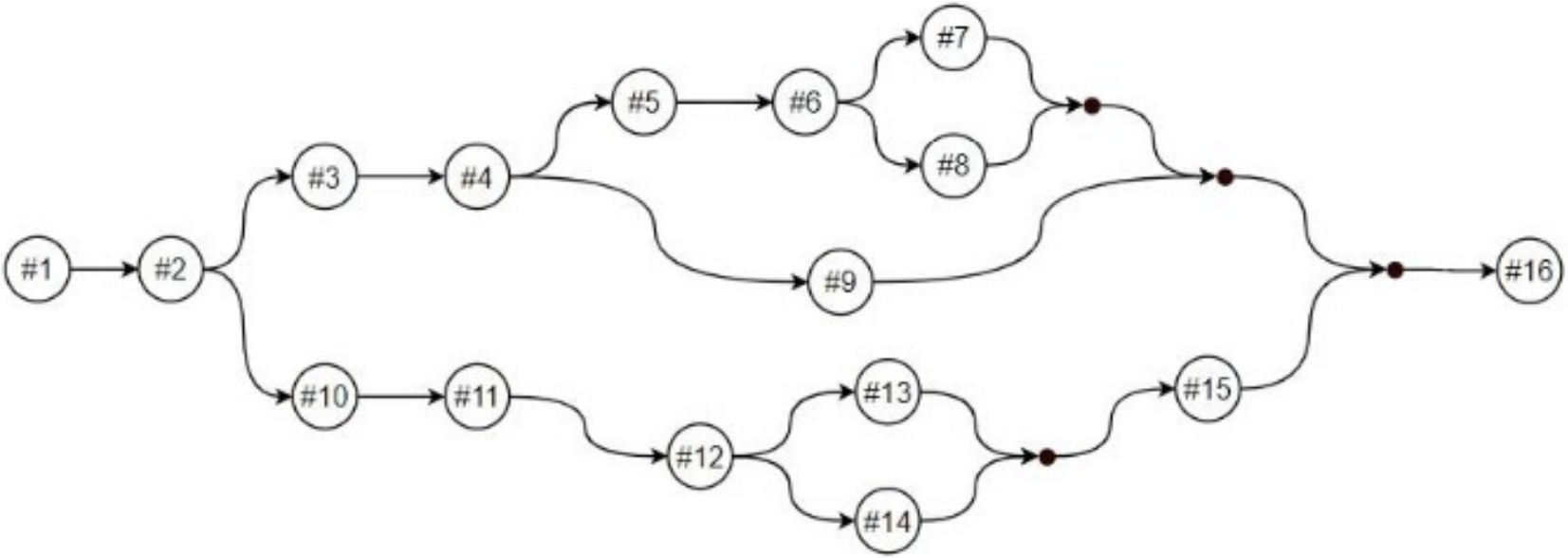
1. O bloco de código é introduzido por dois pontos
2. Instruções pertencentes ao bloco estão no mesmo nível de indentação entre si (4 espaços) relativamente à instrução que introduz o bloco

Estruturas de seleção aninhadas

```
idade = int(input('Qual sua idade? '))     #1
if idade >= 18:                            #2
    cnh = input('Você tem CNH? (s/n)')     #3
    if cnh == 's':                         #4
        cnh_valida = input('Ela está na validade? (s/n)') #5
        if cnh_valida == 's':                             #6
            print('Você pode dirigir!')                   #7
        else:
            print('Você precisa renovar sua CHN!')        #8
    else:
        print('Você precisa tirar uma CNH para dirigir')  #9
else:
    print('Você ainda não pode dirigir!')      #10
    tempo_faltante = 18 - idade                #11
    if tempo_faltante <= 2:                    #12
        print('Falta pouco tempo para que você possa dirigir') #13
    else:
        print('Ainda vai demorar para que você possa dirigir!')   #14
    print('Você pode dirigir um kart se quiser!') #15
print('Fim!') #16
```

Estruturas de seleção aninhadas

Exercício: Com calma, reflita veja se consegue relacionar o programa anterior (olhe a numeração das linhas) com o seguinte fluxo de execução:



Estruturas de seleção encadeadas

São um subtipo das estruturas aninhadas. Dizemos que uma estrutura de seleção A está encadeada com uma estrutura de seleção B quando A está dentro do bloco else de B, e A é a única instrução desse bloco.

Quando isso ocorre, as seleções estão encadeadas entre si, ou seja, cada seleção de encadeamento só é analisada se todas as anteriores resultarem em False.

No instante em que uma das condições é avaliada True, seu bloco é executado e as demais condições serão ignoradas.

Estruturas de seleção encadeadas

Exemplo

```
#Programa que lê a nota de um aluno em uma escala de 0 até 10 e
# converte-a para uma escala usando letras.

nota = float(input('Entre com a nota: '))

if nota >=9 :
    letra = 'A'
else:
    if nota >= 8:
        letra = 'B'
    else:
        if nota >= 6:
            letra = 'C'
        else:
            if nota >= 4:
                letra = 'D'
            else:
                letra = 'E'
print(f'Sua nota escrita como letra é: {letra}')
```

O comando elif

O comando elif é uma fusão dos comandos else e if quando se encontram em uma estrutura de seleção **encadeada.**

```
if <condição 1>:        #1
    <bloco 1>
else:                   #2
    if <condição 2>:    #3
        <bloco 2>
    else:               #4
        <bloco 3>
```

```
if <condição 1>:    #1
    <bloco 1>
elif <condição 2>: #2 + #3
    <bloco 2>
else:               #4
    <bloco 3>
```

Codificação 5.21: Estrutura de seleção encadeada sem `elif` (à esq.) e com `elif` (à dir.).

O comando elif

- O comando elif começa com um else, logo é **sempre** complemento de um if ou de outro elif.
- Não pode existir isoladamente.
- O comando elif termina com um if, e portanto podemos complementá-lo com um else ou outro elif na sequência indefinidamente
- O comando else sempre encerra uma estrutura de seleção, seja ele usado após um if ou elif.
- Com esse encadeamento, o bloco executado será aquele que estiver dentro da primeira estrutura de seleção que resultar em True.

Estruturas de seleção encadeadas

Problema anterior com elif.

```
nota = float(input('Entre com sua nota: '))

if nota >= 9:
    letra = 'A'
elif nota >= 8:
    letra = 'B'
elif nota >= 6:
    letra = 'C'
elif nota >= 4:
    letra = 'D'
else:
    letra = 'E'

print(f'Sua nota convertida para letra é: {letra}')
```

Exercícios Apostila

1. Crie um programa que solicite um número de 1 até 7 e exiba o dia da semana correspondente. Assuma que a semana começa no domingo (1) e termina no sábado(7).

Use apenas seleção simples, ou seja, sem else e nem elif.

2. Refaça o problema anterior, agora utilizando seleção encadeada, mas sem usar o elif, de modo que seja observada a importância do alinhamento correto de indentação entre os diveros comandos if e else

3. Refaça o problema anterior usando elif.

Exercício para casa

Utilizando o que aprendeu nas duas últimas aulas,

resolva novamente o problema de encontrar as raízes de uma equação do segundo grau, sofisticando seu código anterior.

FIM